CSCI4511W Final Project

Cole Mische Jackson Kirchner

Abstract

This project aims to explore machine learning using the Godot game engine. Two agents were trained to play tag under varying environments and reward conditions until they reached approximate Nash equilibria. The trained agents were then pitted against each other to compare their performances, and to see what environment and conditions created the most robust and successful agent. The end result was that agents with more specific reward parameters were often more successful than their non-specific counterparts. They moved in more intelligent ways, and created more interesting games.

Project Description

Pursuit-evasion games are defined as having one or more pursuer agents coordinating their movements to successfully locate and intercept one or more evader agents. This project focuses on a constrained version of this problem, with just two agents. More specifically, we will be running a zero-sum game of tag between a seeker and a runner developed within the Godot Game Engine. In this environment, every gain for the pursuer means a loss for the evader. This ensures both agents must continuously adapt their strategies to improve their reward score.

The technical problem is defined by the interaction between the physical environment and the neural network architectures controlling the agents. The seeker is tasked with minimizing the distance to the runner until a tagging event occurs, while the runner is tasked with maximizing its survival time within the arena. This requires the agents to process parts of their environment, such as relative positions and walls, and translate them into continuous movement actions. Achieving this coordination is not a matter of simple pathfinding; it requires the agents to develop an understanding of the environment and be able to predict the opponent's behavior.

Theoretical and Practical Interest

Pursuit-evasion is particularly interesting to researchers because it roughly mimics many real-world tasks. When researching for this project, we found many papers covering pursuit-evasion as it relates to autonomous drones or missile guidance systems for military usage. While a game of tag may be a fairly innocuous abstraction, a lot of the core principles

can still apply to these real-world examples. But beyond these applications, the problem is intellectually stimulating due to the concept of the Nash Equilibrium. This is the point in training where both agents have optimized their policies such that neither can improve their outcome by changing their strategy alone. Reaching this state in a 1v1 tag game reveals the limits of the chosen algorithms and the effectiveness of the reward structures provided to the agents.

The complexity of the problem is also increased when it comes to generalization. Agents might learn how to play tag in a specific room, but won't learn how to apply the more abstract concept of pursuit. Creating agents that are robust enough to handle these environmental changes is one of the main goals of our project and represents a significant step toward creating more versatile artificial intelligence.

Implementation

This project utilizes the Godot Reinforcement Learning Agents library to bridge the gap between high-level Python training scripts and the low-level physics of the game engine. The Godot game is started, then the Python script to begin the machine training is started. A custom sync node is implemented in Godot to allow the two to communicate through a TCP socket. Python "plays" the game in the running environment, and will output ONNX files. These files represent trained agents that can be plugged back into Godot without external Python running. This is particularly useful for examining how agents trained in different environments compare to one another in new environments.

Finally, the project addresses the "sparse reward" problem through the implementation of curriculum learning and shaped rewards. In many scenarios, an agent may struggle to learn if it only receives a reward upon making physical contact, as that event may be extremely rare during early training. By adjusting the tagging radius dynamically or rewarding the pursuer for cutting off the evader's path, we can guide the agents through a more efficient learning process. This methodological approach not only speeds up data collection but also provides a clearer window into how specific reward variables influence the final "intelligence" of the agents.

Literature Review

Infrastructure of the Project

Perhaps the most vital component of a project like this is the software in which everything is built. Edward Beeching and his team have created the *Godot Reinforcement Learning Agents* library that can enable this training with just a small amount of set up. It is completely open source, and the documentation serves as a key resource in the creation of this project [1].

This library allows us to use Godot, an open-source game engine, to train agents using Python. In the words of Beeching: "[The RL-agents library] is a flexible, fast and robust interface between RL training code running in Python and the Godot environment,"

It uses a custom *Sync* node in Godot which creates a TCP connection between the game an AI trainer, passing observations and rewards back and forth through a TCP socket. The library also uses the fairly common gym framework OpenAI in 2016. Agents receive structured feedback based on their performance in the environment [2].

Algorithms to Consider

Since our project only has two agents, it will be a zero-sum game, meaning there can only be a single winner, while the other agent is the loser. A lot of recent research suggests that the Proximal Policy Optimization (PPO) algorithm is a strong contender for this kind of setup since we don't want an agent to take a massive leap in logic in an agent versus agent scenario like this. The PPO algorithm is an algorithm that prevents learning agents from making early conclusions about a problem. An example of this would be if an agent made a bad decision, the agent could jump to the conclusion that there's no nuance about the decision. PPO forces the agent to try these decisions again but with some variance to see if there's a possible solution around that decision.

PPO is our best bet when it comes to complex algorithms for reinforced learning [3]. It is very reliable, and won't require a lot of maintenance or tweaking since PPO prevents the agents from getting confused by new situations. For a game like tag, which is a 1v1 game, it means the agents will constantly look for ways to evade/pursue the other. This will force the agents to find smarter and better ways to outsmart each other, and the metagame will continue to evolve.

Something to consider during run-time is the Nash Equilibrium, which is the point where both agents are perfectly optimized for the game. The "Equilibrium Policy Generalization" shows that agents will be trained to find these specific strategies [4]. In simpler environments, this equilibrium will likely be fairly quick to find.

Another important aspect of our project is to include multiple different levels in which the agents can play against each other. This could pose a challenge, because an agent that learns to play on one map might fail completely on a different one. This is often called a lack of generalization [5]. To counter this, we can take inspiration from the framework proposed by Chen et al. [6]. In their training, they end up using an "adaptive environment generator," which dynamically creates different levels for agents to work in. This ensures that the agents don't just memorize one room but actually learn how to play the game in any environment.

Unfortunately, we could end up with rooms where the pursuer brainlessly follows the evader in hopes of tagging them. To avoid this situation, the pursuer could be rewarded not only for tagging the evader, but also moving to where the tagger is currently moving toward. This would help the AI learn to cut off the evader's path instead of just following behind. This would ultimately help promote generalizability and robustness for the pursuer.

A similar framework to the adaptive environment generator exists called Equilibrium Policy Generalization [4]. Essentially, this involves random addition/removal of graph nodes during run-time to induce randomness. While our game of tag doesn't involve graphs/nodes, perhaps we could take inspiration from it and dynamically add walls/obstacles as during

run-time of the training.

A problem we might face during training is difficulty with giving rewards; teaching an agent to catch an evader might be hard if we only reward when they actually touch. This is known as a sparse reward problem. To fix this, Xu et al. suggests using what they call curriculum learning: "The core principle of this CL strategy is to treat the terminal capture conditions not as fixed constraints during training but as adjustable parameters of the learning problem itself." [7]

This idea would be very easy to implement in a game of tag. For example, the tagging radius of the pursuer could start as very large. As the agent improves, the radius gets smaller until it matches the actual rules of the game. Conversely, the evader could be rewarded by maintaining distance from the pursuer under a similar system.

Due to the amount of time needed to train AI models, we need to optimize the way we train and collect data. The article "Optimizing Data Collection in Deep Reinforcement Learning" suggests that we can run multiple simulations all at once to speed up the model training process [8]. By running multiple simulations at the same time, we can speed up the model training and collect more data. Potentially useful data could include evader win rate, game duration, common tagging locations, proximity to obstacles, etc. By tracking all this data, we can properly balance and fine tune the agents to reach optimal playstyles and strategies in our simulation of tag.

Overall, these articles have given us a lot to think about when it comes to implementing our software. It has given us ideas on how to make more adaptive, full-bodied agents that should be more suited to perform across a variety of environments. Algorithms such as PPO can ensure the agents develop properly without making any leaps in logic during the learning process. Implementing an adaptive environment will help the agents deal with new levels. Precisely tuning our rewards system can help train the agents more efficiently. And diligent data collection can reveal insights that might not be obvious at first glance, as well as help inform the agents better. While we may not implement every idea presented, this will give us a much stronger start in what we aim to accomplish with the project.

Approach

Our goal is to run a series of controlled experiments to see how different training conditions affect agent behavior. Instead of just creating one smart agent, the project creates several versions of the seeker and runner. These versions are trained with different reward sets to see which set creates the most capable AI. We also sought to create less intelligent versions of the AI by using lackluster reward systems. For the neural network, we utilized the script featuring PPO from the Godot Reinforcement Learning Agents library. This means our neural network takes advantage of PPO, making PPO a part of the foundation of the training.

Variations with Reward Mechanism

The first part of the approach involves changing how the agents are rewarded for their actions. This is done to solve the problem of agents not learning because the goal of tagging is too difficult to reach by accident. The project uses three distinct reward profiles to compare learning efficiency:

- **Sparse Reward Group:** The seeker only receives a reward when it physically touches the runner, and the runner only receives a reward for avoiding the seeker until the timer expires. This setup is expected to produce the worst results, but it will provide a baseline to show why more complex reward systems are necessary.
- **Distance Group:** This group will be rewarded based on if the distance between the agents is growing or shrinking. If the distance between the two agents is actively shrinking, the tagger will be rewarded and the evader will be punished. This will incentivize the evader to always be moving away from the evader under any circumstances.
- **Complex Group:** These agents are able to use rays to "see" their environment more efficiently using rays to detect anything that has collision. This will be helpful for preventing the AIs from stepping into local maximum.

Data Collection & Comparison

The final stage of the approach is the comparison of the data for these different groups. We recorded the average win rate, average time taken for each successful tag, and average distance between the agents across 2,000 rounds of tag. We used variables in Godot to keep precise track of the environment variables, then wrote everything to CSV files for analysis. By comparing these numbers across all the different agent groups, the project can reveal exactly how different reward structures and obstacles influence the final intelligence and strategies of the AI. Further conclusions are drawn later in the analysis section.

Results

For a more qualitative look at the agents' performance, please watch this demo video we recorded which is a compilation of clips from the different training processes. Also note, if one wishes to see our exact data they can view the CSV files in the GitHub repo. Below are the graphs recorded for each set of AI agents.

Data for sparse reward AIs:

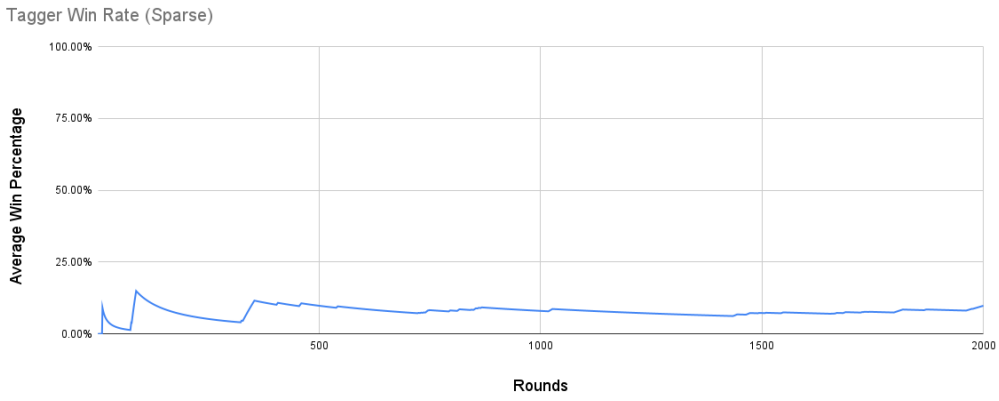

Figure 1: Tagger Win Rate (Sparse)

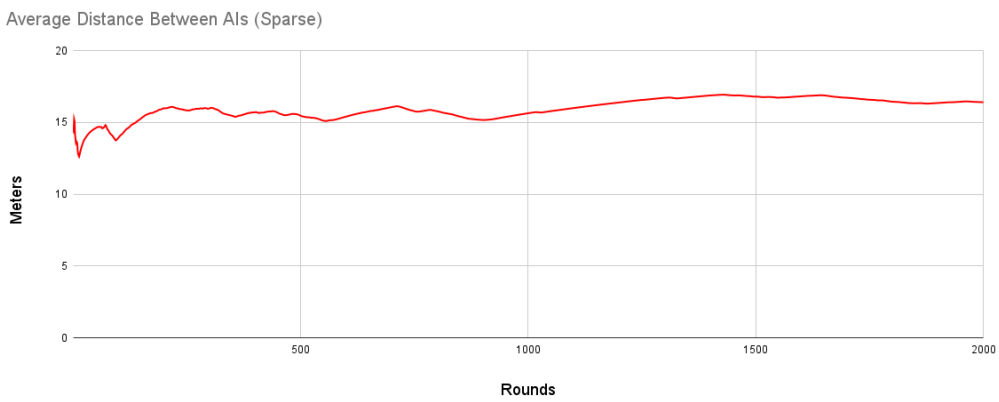

Figure 2: Average Distance Between AIs (Sparse)

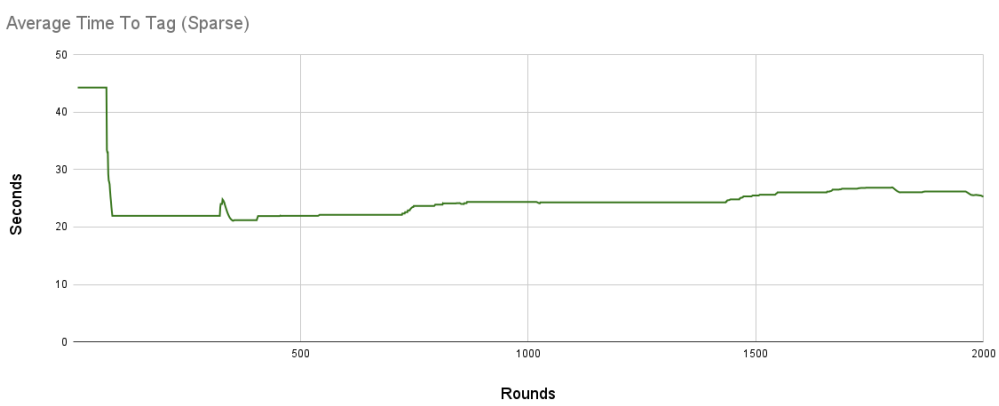

Figure 3: Average Time To Tag (Sparse)

Data for Distance-based reward AIs:

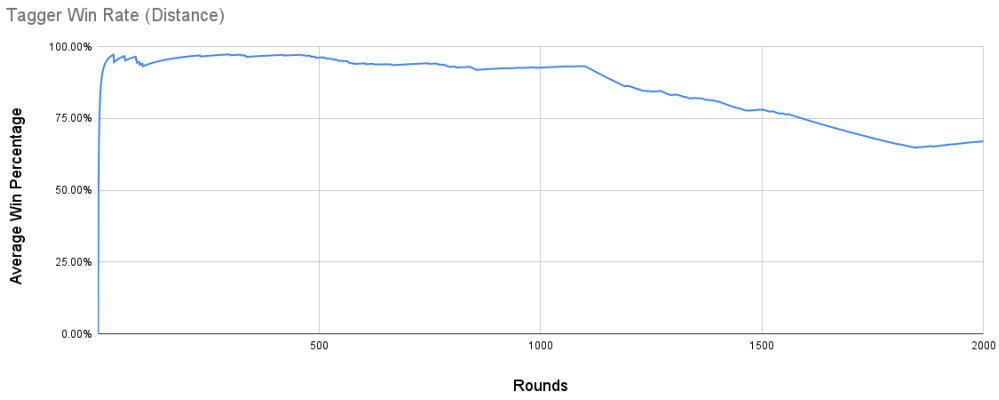

Figure 4: Tagger Win Rate (Distance)

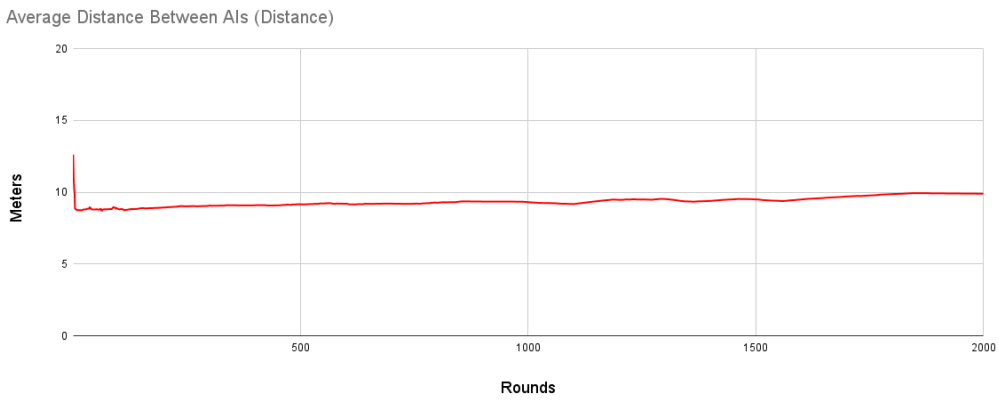

Figure 5: Average Distance Between AIs (Distance)

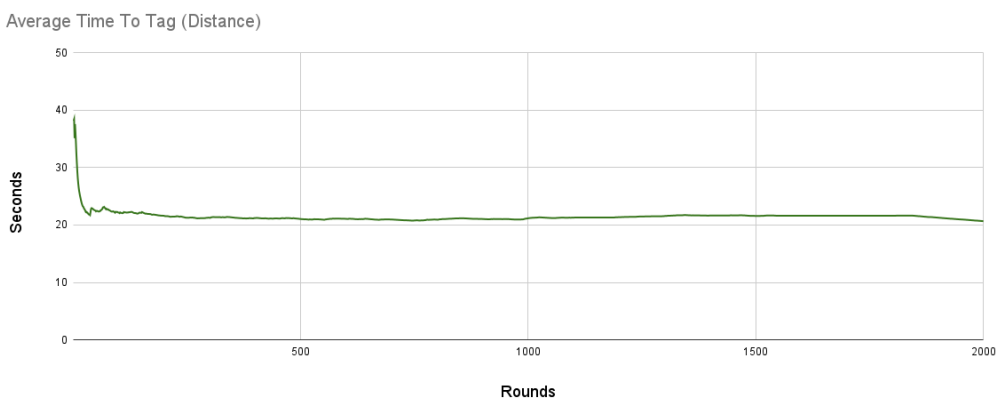

Figure 6: Average Time To Tag (Distance)

Data for Complex reward system AIs:

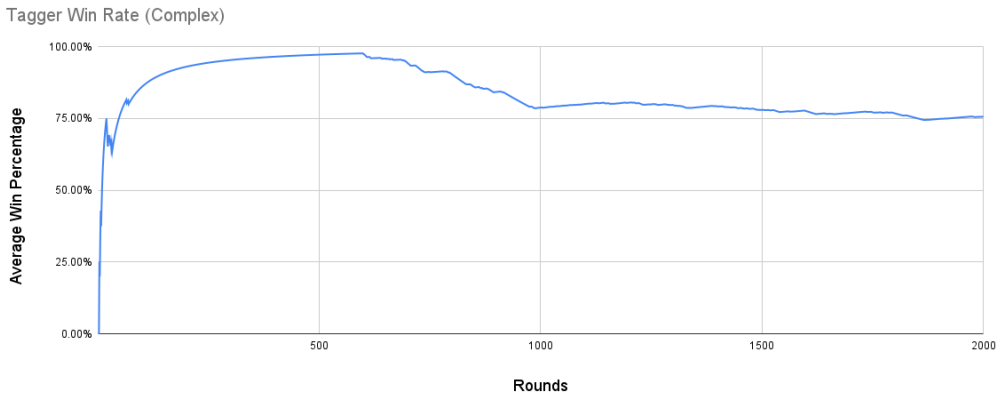

Figure 7: Tagger Win Rate (Complex)

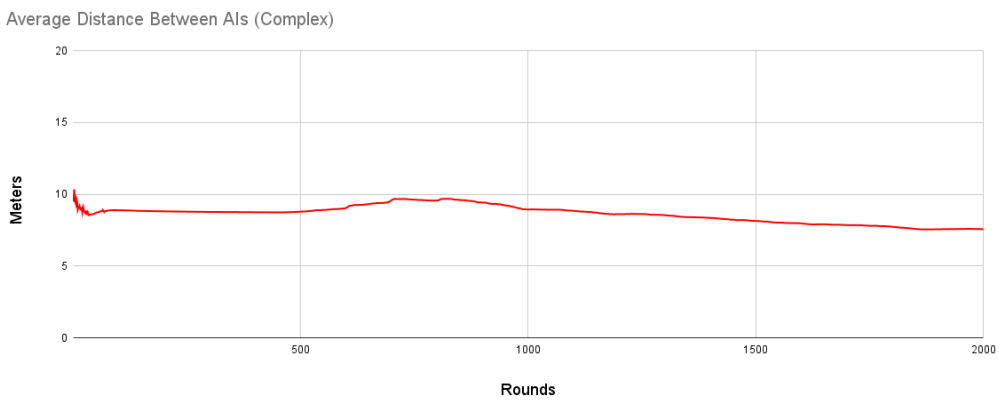

Figure 8: Average Distance Between AIs (Complex)

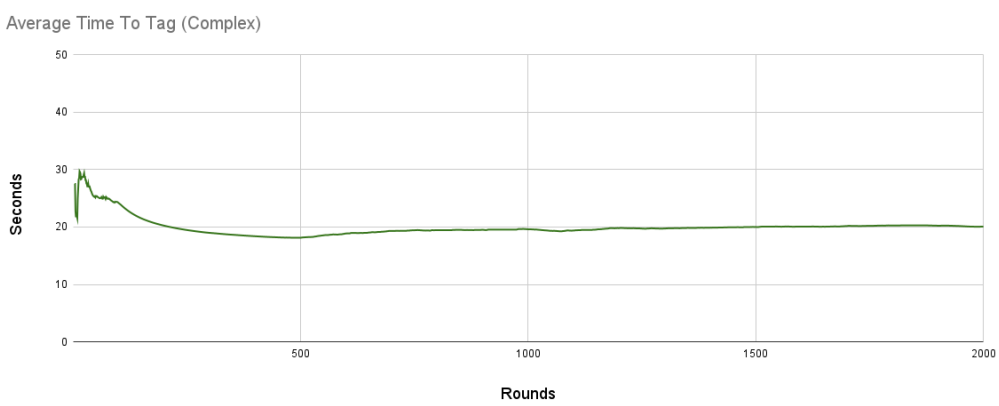

Figure 9: Average Time To Tag (Complex)

Analysis

The results of the training sessions provide a clear comparison of how different reward structures impact the learning process and final behavior of reinforcement learning agents. Analyzing the data from the different reward groups clearly shows the effects of more precise reward systems. They lead to faster convergence toward the Nash equilibrium and more effective pursuit strategies.

Sparse Reward Model

The agents trained with the sparse reward structure faced the most significant challenges due to the sparse reward problem. Because the seeker was only rewarded for a successful tag, it had no guidance on how to actually find the runner in the early stages of training. For the first several rounds, the tagger maintained a 0% win rate as it essentially moved at random within the arena. For all intents and purposes, the agents may as well have been searching in the dark for their targets. The tagger would only be rewarded when they reach the evader, but since the evader consistently moves, the tagger has a very tough time actually reaching it. Imagine you're given 45 seconds to find a moving banana in a room while blindfolded. That's what the tagger had to do, with this very basic reward system.

After 2,000 rounds of training, the tagger only reached an overall win rate of 9.8% percent. The average distance between the agents remained the highest of all three groups, ending at over 16.4 meters. This suggests that the seeker never actually learned to pursue the runner. Instead, the few tags that did occur were simply the result of the two agents accidentally colliding with each other. The lack of secondary rewards prevented the seeker from developing any understanding of the spatial relationship between itself and its target.

Distance-Based Model

The Distance reward group showed a massive improvement over the sparse reward model by introducing continuous feedback. Rewarding the seeker for closing the gap allowed it to understand the goal of the game after just a handful of rounds (visible with the early spike in the graph). The tagger started to home in on the evader aggressively, pushing it to the corner every time. However, as the games went on, the evader stopped moving to the corner immediately. This became a huge problem for the tagger, as it was almost ingrained into it to dive into the evader side's corners. With a higher level of entropy in the neural network, perhaps it could've adapted sooner instead of getting caught in a local maximum of behavior.

This model ended with a tagger win rate of 67.05%. The average distance between agents was also much lower, staying around 9.9 meters compared to the first model. This lower distance was because the seeker was actively following the runner throughout the match. Additionally, the average time taken to tag dropped to 20.7 seconds, showing that the tagger wasn't just fumbling around in the dark, but actually pursuing the evader.

Complex Reward Model

The "Complex" reward group performed the best out of all three test cases. This model is built on utilizing raycasts for the evader to be able to see into the world. The evader gets a negative reward for colliding with any walls or being near the tagger. The evader is also rewarded each frame that it is alive. The tagger gets rewarded for attempting to predict the movement of the evader. This created a sense of urgency that forced the seeker to be even more aggressive. By the end of the training, the complex seeker reached the highest win rate of the three groups at 75.5%.

The impact of the extra penalties is most visible in the proximity and timing data. The average distance between agents in this group was the lowest recorded at only 7.56 meters. This is because the evader developed a strategy of playing footsies with the tagger. The runner would try to fake out the seeker, while the seeker was trying to predict the runner's movement. The average time to tag also reached its lowest point at 20.07 seconds. While this was only slightly faster than the distance model, the combination of a higher win rate and lower average distance shows how the seeker became dominant in its strategy to cut off the movement of the runner.

Comparative Summary

When looking at all the data together, there is a clear hierarchy of performance based on the complexity of the reward function. The sparse reward model confirms that agents cannot learn complex tasks like pursuit-evasion without some form of guided feedback. Moving from the sparse reward to the distance model resulted in a win rate increase of over 500 percent and a significant reduction in the distance between agents.

The transition from the distance model to the complex model provided the final layer of optimization. While the distance model taught the agent how to follow the target, the complex model taught it to do so with speed and precision. The final win rates of 9.25% for the stupid model, 66.85% for the distance model, and 75.56% for the complex model prove that specific reward parameters are essential for creating successful agents. This data supports the idea that the design of the reward system is just as important as the algorithm used for training.

Conclusion

Overall, this project showed us that how you design a reward system is just as important as the actual training algorithm. The sparse reward model was basically random because the tagger just wandered around blindly and only won about 9% of the time by accident. Moving to distance-based rewards changed everything, making the win rate jump up to 67% because the tagger was incentivized to chase. Our complex model was definitely the best though, hitting a 75.5% tagger win rate by using raycasts to see and learning to predict where the runner was going.

If we were to continue iterating on this project, there's a few things we could do:

- We could add more complex movement systems such as dashing or jumping to create a more complex environment and more interesting games.
- We could test the agents in more levels, and try exporting the AIs to different levels to see how they perform.
- Continue to refine the reward system and run further testing to have the agents continue to evolve the metagame.
- Running the trials for many more rounds could yield interesting results. We ran the simulations thousands of times, but real life applications can reach into the millions.

Nonetheless, it was very interesting to see these agents learn and grow in real time. It was captivating to see them reach a sort of Nash equilibrium where they both optimized their strategies so much that neither could get an edge without the other changing something first. They even started playing "footsies" and actually tried to outsmart each other rather than just running in straight lines.

Bibliography

References

[1] Edward Beeching, Jilles Debangoye, Olivier Simonin, and Christian Wolf. Godot reinforcement learning agents. *arXiv preprint arXiv:2112.03636*, 2021.

[2] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

[3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[4] Runyu Lu. Equilibrium policy generalization: A reinforcement learning framework for cross-graph zero-shot generalization in pursuit-evasion games. *arXiv preprint arXiv:2511.00811*, 2025.

[5] David Bertoin and Emmanuel Rachelson. Local feature swapping for generalization in reinforcement learning. *arXiv preprint arXiv:2204.06355*, 2022.

[6] Jiayu Chen, Chao Yu, Geng Li, Wendong Tang, Shuo Ji, Xinghan Yang, Bo Xu, Huazhong Yang, and Yi Wang. Online planning for multi-uav pursuit-evasion in unknown environments using deep reinforcement learning. *arXiv preprint arXiv:2409.15866*, 2025.

[7] Zhenyang Xu, Shidong Shao, and Zhu Han. A DRL framework for autonomous pursuit-evasion: From multi-spacecraft to multi-drone scenarios. *Drones*, 9(9):636, 2025.

[8] James Gleeson, Danny Snider, Zissis Poulos, Mark Gabel, Eyal de Lara, and Gennady Pekhimenko. Optimizing data collection in deep reinforcement learning. *arXiv preprint arXiv:2207.07736*, 2022.

Group Contributions

Cole Mische

- Set up initial tag environment in Godot
- Wrote project description
- 50% of literature review
- Refactored agents to a strategy-esque design pattern
- Collaborated on data graph creation
- Collected data for sparse game and wrote analysis
- Collected data for distance game and wrote analysis
- Collaborated on final document formatting

Jackson Kirchner

- Configured initial RL setup for tag environment
- Set up proper tag arena
- 50% of literature review
- Collaborated on data graph creation
- Collected data for complex game and wrote analysis
- Wrote code for complex Evader
- Wrote code for distance Evader
- Testing the networks and fine-tuning reward systems
- Collaborated on final document formatting